

Finite Strain Continuum Mechanics Formulation for Large Deformation Torsion in Solid Circular Shafts: A C++ and FEM Comparative Analysis

*Project report submitted
in partial fulfilment of the requirement for the degree of*

Master of Technology

Course code: ME 541 – Continuum Mechanics

By

Name and Roll no.: -

Neel M. Kapadi, 254103109



**DEPARTMENT OF MECHANICAL ENGINEERING
INDIAN INSTITUTE OF TECHNOLOGY GUWAHATI
(April 2026)**

TABLE OF CONTENTS

ABSTRACT	4
1. INTRODUCTION	5
2. CONTINUUM MECHANICS FORMULATION	5
2.1 KINEMATICS AND DEFORMATION GRADIENT	5
2.3 MATERIAL MODELS AND STRESS TENSORS	6
3. METHODOLOGY AND IMPLEMENTATION VIA C++ CODE	6
4. ALGORITHM	7
5. NUMERICAL IMPLEMENTATIONS (C++ CODE) RESULTS	8
6. FINITE ELEMENT SIMULATION SETUP	8
6.1 MESH CONVERGENCE STUDY	18
7. RESULTS AND DISCUSSIONS	19
7.1 COMPARISON AND VALIDATION	19
7.2 DISCUSSION AND PHYSICAL INTERPRETATION	23
8. CONCLUSION	24
9. REFERENCES	24
APPENDIX A: NUMERICAL ITERATION CODE RESULTS	25
APPENDIX B: C++ SOURCE CODE	25

LIST OF FIGURES

Figure 6.1 Material selection from Engineering data	9
Figure 6.2 Generation of 3D model	10
Figure 6.3 10mm mesh generated on shaft	10
Figure 6.4 7.5 mm mesh generated on shaft	11
Figure 6.5 5 mm mesh generated on shaft	11
Figure 6.6 Fixed support at one end and Rotation at another end	12
Figure 6.7 Total Deformation on 5mm shaft	12
Figure 6.8 Total Deformation on 7.5mm shaft	13
Figure 6.9 Total Deformation on 10mm shaft	13
Figure 6.10 Max Shear Stress on 5mm shaft	14
Figure 6.11 Max Shear Stress on 7.5mm shaft	14
Figure 6.12 Max Shear Stress on 10mm shaft	15
Figure 6.13 Shear Stress along radial line	15

Figure 6.14 Axial normal stress	16
Figure 6.15 Normal Stress along radial line	16
Figure 6.16 Torque generated on shaft for 5mm mesh	17
Figure 6.17 Torque generated on shaft for 7.5mm mesh	17
Figure 6.18 Torque generated on shaft for 10mm mesh	18
Figure 7.1 Error between Numerical and Analytical Deformation Gradient	20
Figure 7.2 Axial Normal stress v/s radius	21
Figure 7.3 Torsional Shear Stress v/s Radius	22
Figure 7.4 Torque v/s Rate of Twist	23

LIST OF TABLES

Table 5.1 Summary of results generated from code	8
Table 6.1 FEM Mesh Convergence	18

ABSTRACT

This report presents a computational and theoretical analysis of the torsional behaviour of a solid circular shaft undergoing large rotations. A custom C++ script was developed to evaluate the breakdown of classical small-deformation linear elasticity by comparing it directly with a finite strain continuum mechanics formulation. The implemented algorithm successfully evaluates the deformation gradient analytically and numerically, calculates the Green-Lagrange strain, and computes both the Second Piola-Kirchhoff and Cauchy stress tensors. To validate the custom computational framework, a finite element simulation was subsequently conducted using Ansys Static Structural. The comparative results emphasize the strict necessity of finite strain theories to accurately capture geometric nonlinear phenomena under severe twist. Specifically, the finite strain model successfully captures the Poynting effect—the physical tendency of the shaft to develop axial normal stresses when subjected to large rotations—which is completely ignored by classical torsion theory. Ultimately, this study proves that classical linear assumptions and relations are fundamentally insufficient for predicting the true stress state of shafts subjected to severe rotational deformation.

Keywords: Finite Strain, Continuum Mechanics, Large Rotations, Poynting Effect, Finite Element Simulation

1. INTRODUCTION

The torsion of a solid circular shaft is a foundational problem in solid mechanics. Classical torsion theory assumes infinitesimal deformations, predicting a purely linear shear stress distribution ($\tau(r)=G\alpha r$) and concluding that no axial stresses are generated during twisting. However, when a shaft undergoes a large rotation—such as the $\alpha = 5.0$ rad/m twist applied in this computational model—the small angle assumptions become invalid, and geometric nonlinearities dominate the material's response. The primary objective of this project is to implement a finite strain continuum mechanics formulation via a C++ programming approach, extract local stress and strain measures, and systematically compare these finite deformation findings against classical small deformation predictions.

2. CONTINUUM MECHANICS FORMULATION

To accurately model the large rotation of the shaft, the deformation is defined in a cylindrical coordinate system as $r = R, \theta = \Theta + \alpha Z, z = Z$

2.1 KINEMATICS AND DEFORMATION GRADIENT

Mapping this to Cartesian coordinates yields the deformation map:

$$x = X \cos(\alpha Z) - Y \sin(\alpha Z)$$

$$y = X \sin(\alpha Z) + Y \cos(\alpha Z)$$

$$z = Z$$

The analytical deformation gradient $\mathbf{F} = \partial \mathbf{x} / \partial \mathbf{X}$ is computed as:

$$\mathbf{F} = \begin{bmatrix} \cos(\alpha Z) & -\sin(\alpha Z) & -\alpha y \\ \sin(\alpha Z) & \cos(\alpha Z) & \alpha x \\ 0 & 0 & 1 \end{bmatrix}$$

Within the C++ implementation, this is compared against a numerically derived deformation gradient using the **central finite difference method** to evaluate algorithmic accuracy.

2.2 STRAIN MEASURES

To contrast classical and finite strain theories, two distinct strain measures are computed:

- **Infinitesimal Strain Tensor (ϵ):** Used for linear elasticity, defined as

$$\epsilon = \frac{1}{2}(\nabla u + (\nabla u)^T)$$

- **Green-Lagrange Strain Tensor (E):** Used to capture geometric nonlinearity, defined as

$$E = \frac{1}{2}(C - I), \text{ where } C = F^T F \text{ is the right Cauchy-Green deformation tensor.}$$

2.3 MATERIAL MODELS AND STRESS TENSORS

For Model 1 (Small Strain), the Cauchy stress is computed using Hooke's law:

$$\sigma^{(s)} = \lambda \text{tr}(\epsilon)I + 2\mu \epsilon$$

For Model 2 (Finite Strain), the Second Piola-Kirchhoff stress (S) is computed and then pushed forward to the spatial configuration to obtain the Cauchy stress $\sigma^{(f)}$:

$$S = \lambda \text{tr}(E)I + 2\mu E$$

$$\sigma^{(f)} = \frac{1}{J} F S F^T$$

The torsional shear stress ($\sigma_{z\theta}$) and axial normal stress (σ_{zz}) are extracted from these tensors. The total reaction torque is then calculated by numerically integrating the shear stress over the cross-sectional area:

$$T = \int_A r \sigma_{z\theta} dA$$

3. METHODOLOGY AND IMPLEMENTATION VIA C++ CODE

The problem considers a straight circular shaft defined by a radius $R=0.05$ m, a length $L=0.5$ m, Elastic Modulus $E=200$ GPa, and Poisson's ratio $\nu=0.3$. The reference configuration utilizes material coordinates (X, Y, Z) . The methodology is driven by a custom C++ implementation divided into three main analytical approaches:

- Continuum Mechanics C++ Code:** The code sweeps through a grid of material points ($nR=20$, $n\theta=36$) and maps them to their deformed spatial coordinates. The deformation gradient (F) is calculated two ways: exactly, using the `getAnalyticalF` function, and computationally, using a central finite difference method within the `getNumericalF` function (utilizing a

perturbation step of ($\delta=10^{-6}$). The Second Piola-Kirchhoff stress (S) is formulated as $S=\lambda tr(E)I+2\mu E$. To evaluate spatial tractions, the program pushes this tensor forward to obtain the Cauchy stress tensor $\sigma^{(f)} = \frac{1}{J}FSF^T$.

ii. Stress Extraction & Integration: A dedicated C++ routine, `extractStressComponent`, is utilized to isolate the torsional shear stress ($\sigma_{z\theta}$) and the axial normal stress (σ_{zz}) from the 3×3 Cauchy stress tensor array. The total reaction torque (T) is subsequently computed by numerically integrating the shear stress across the shaft's cross-sectional area.

iii. Classical Theory Comparison: The C++ code directly computes the analytical baselines using the `getClassicalShearStress` and `getClassicalTorque` functions. These are compared against the finite strain outputs to evaluate the limitations of the linear assumptions.

4. ALGORITHM

The logical flow of the C++ program executes the following steps to populate the analytical dataset:

- i. Initialization:** Define material constants (E,v) and geometric parameters (r,L, α) in the `main()` execution block. Initialize matrix utility functions like `identityMatrix()`, `addMat()`, and `subMat()`.
- ii. Discretization:** Loop over the cross-section to generate a grid of points spanning the radius and circumference of the shaft.
- iii. Kinematics:** Apply the large-rotation deformation map to calculate the deformed coordinates.
- iv. Deformation Gradient Assembly:** Compute the exact analytical tensor (F) via `getAnalyticalF` and the numerical tensor (Fnum) via `getNumericalF`.
- v. Strain and Stress Calculation:** Formulate the Green-Lagrange strain (E). Use material properties (λ,μ) to find S and push it forward to derive the Cauchy stress (σ).
- vi. Component Extraction:** Pass the calculated Cauchy stress into the `extractStressComponent` function to filter out the torsional shear stress (τ_{finite}) and the axial normal stress (σ_{zz}).

- vii. Classical Baseline Computation:** Simultaneously calculate $\tau_{\text{classical}}$ and $T_{\text{classical}}$ via `getClassicalShearStress` and `getClassicalTorque` for direct side-by-step comparative analysis.
- viii. Output:** Export all nodal coordinates, F tensors, strain tensors, stress tensors, relative errors, and differential torque contributions to a CSV file (`shaft_torsion_data.csv`) for post-processing and visualization.

5. NUMERICAL IMPLEMENTATIONS (C++ CODE) RESULTS

The complete C++ source code developed for the algorithms is provided in Appendix A. The summary of results generated from C++ code for Shear Stress, Deformation, Torque for Small Strain, Finite Strain and Classical Method for various radius and angle is shown in the table below.

Table 5.1 Summary of results generated from code

Parameters		Deformation: ux, uy, uz (m)			Small Strain			Finite Strain			Classical Strain
Radius (m)	Angle (Rad)				Shear Stress (Pa)	Axial Normal Stress (Pa)	Torque (Nm)	Shear Stress (Pa)	Axial Normal Stress (Pa)	Torque (Nm)	Shear Stress (Pa)
0.00125	0.959931	-0.001904	-0.001415	0	4.81E+08	- 4.16E+11	0.327774	4.81E+08	5.26E+06	0.327796	4.81E+08
0.01875	0.436332	-0.03535	-0.004102	0	7.21E+09	- 4.16E+11	1106.24	7.32E+09	1.18E+09	1123.25	7.21E+09
0.02375	0.087266	-0.043853	0.010431	0	9.13E+09	- 4.16E+11	2248.2	9.36E+09	1.90E+09	2303.68	9.13E+09
0.03875	0.959931	-0.059029	-0.04387	0	1.49E+10	- 4.16E+11	9764.71	1.59E+10	5.05E+09	10406.2	1.49E+10
0.04875	1.309	-0.050907	-0.077263	0	1.88E+10	- 4.16E+11	19443.2	2.07E+10	8.00E+09	21464.8	1.88E+10

6. FINITE ELEMENT SIMULATION SETUP

A finite element simulation was conducted to validate the continuum mechanics code. For modelling of 3D shaft and torsion simulation, Ansys 2026 R1 Static Structural was used. The model consists of a 3D solid cylinder with a radius of $r = 0.05$ m and a length of $l = 0.5$ m. The material was defined as an isotropic elastic solid with an Elastic Modulus of $E = 200$ GPa and a Poisson's ratio of $\nu = 0.3$.

To replicate the theoretical conditions, the boundary conditions were set such that one end of the cylinder was fully fixed, while a prescribed rotation of $\theta = 2.5 \times (180/\pi) \approx 143.24^\circ$ or 2.5 rad was applied about the Z-axis at the opposite end. The geometry was meshed using 3D solid elements and the desired results such as Torque, deformation, shear stress and normal stress are extracted.

The steps mentioned below are used for performing the torsional analysis of 3D shaft using ANSYS software.

A. MATERIAL SELECTION

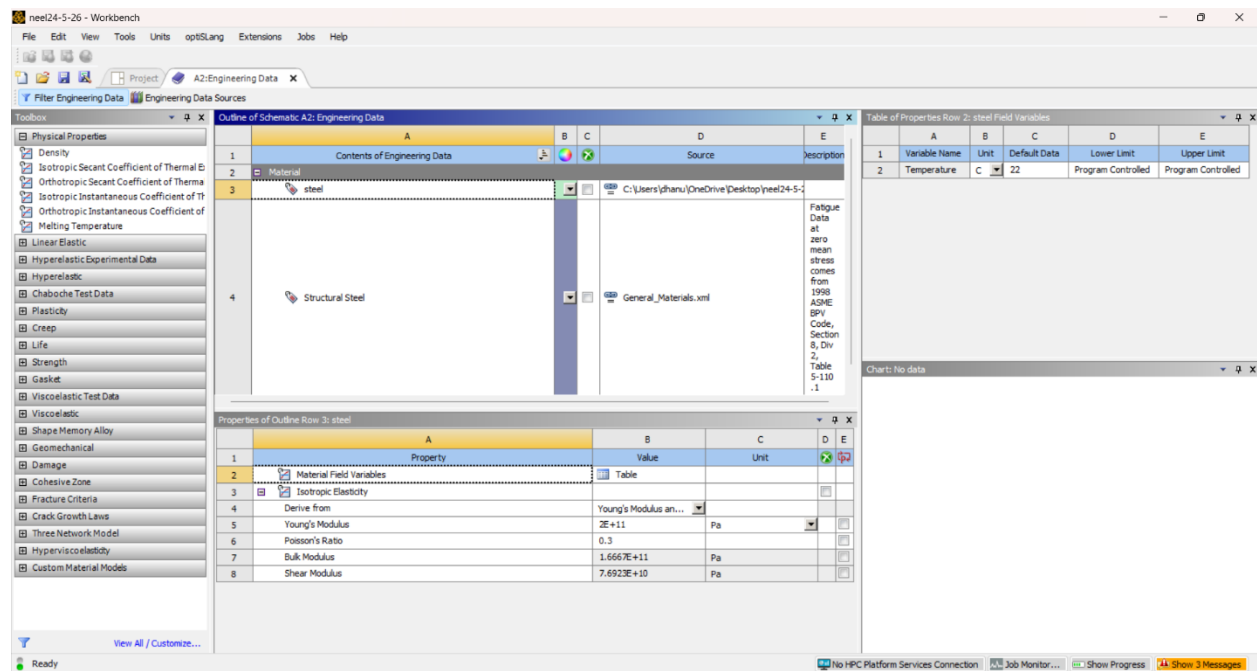


Figure 6.1 Material selection from Engineering data

B. GENERATION OF 3D MODEL IN DESIGN MODELER

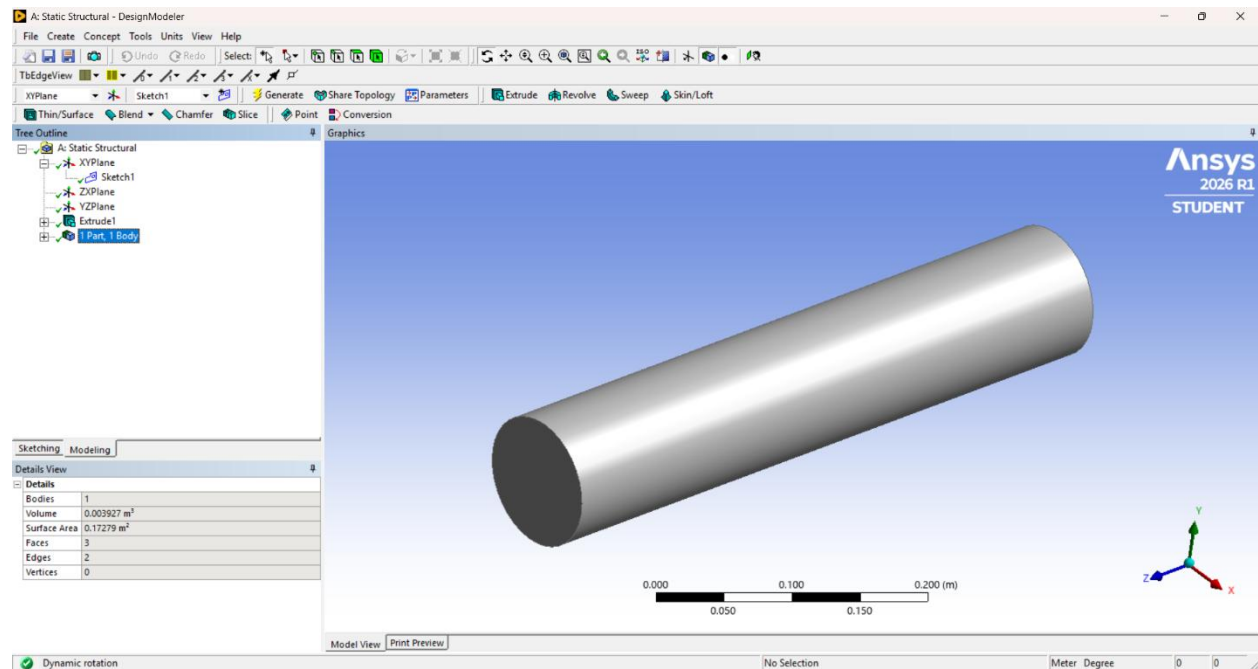


Figure 6.2 Generation of 3D model

C. GENERATE MESH

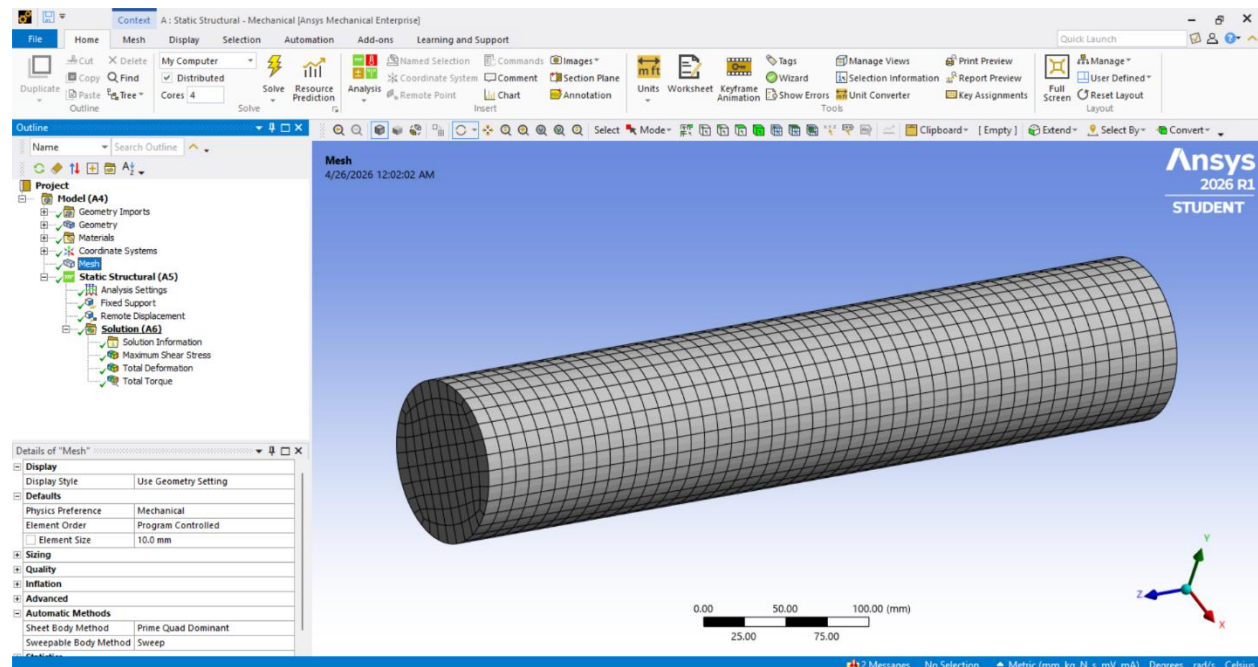


Figure 6.3 10mm mesh generated on shaft

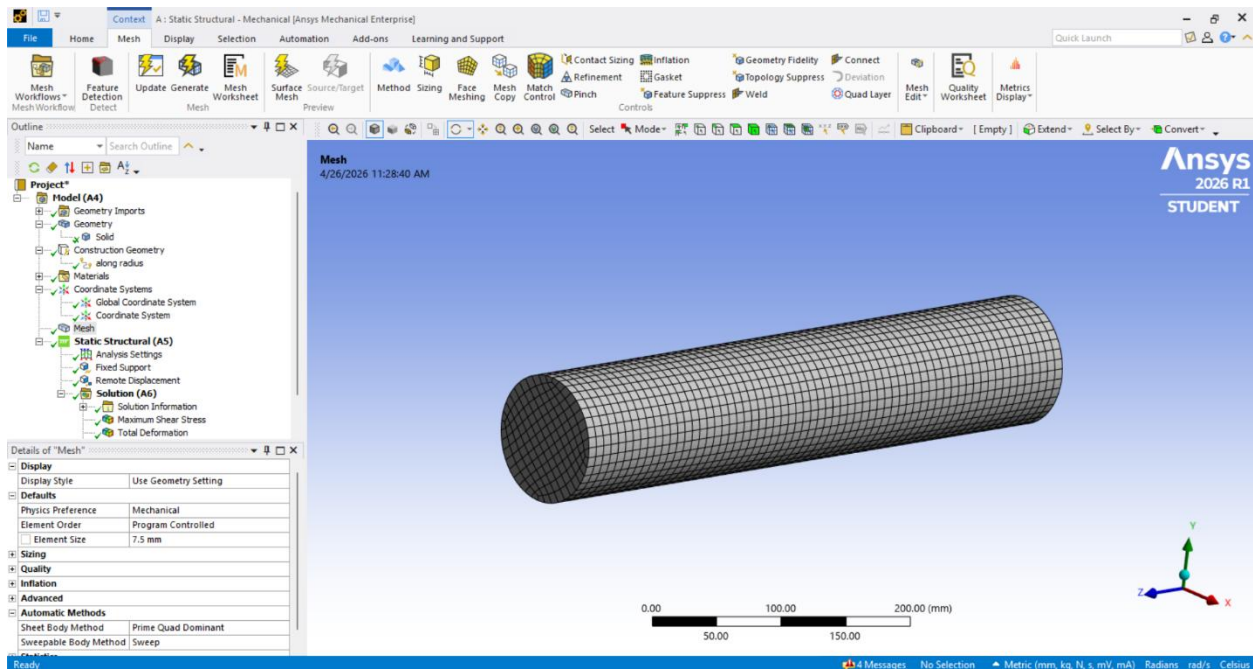


Figure 6.4 7.5 mm mesh generated on shaft

Mesh
4/26/2026 11:45:30 AM

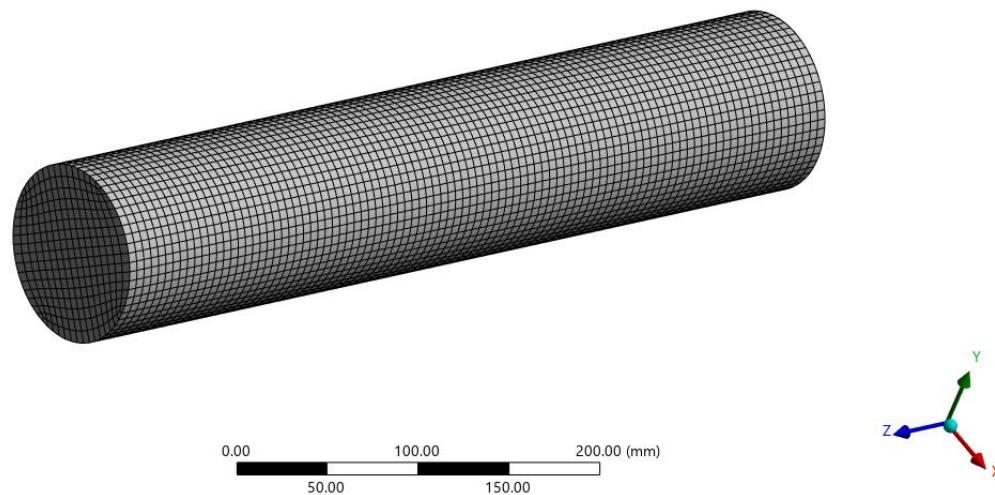


Figure 6.5 5 mm mesh generated on shaft

D. APPLY BOUNDARY CONDITIONS IN SETUP

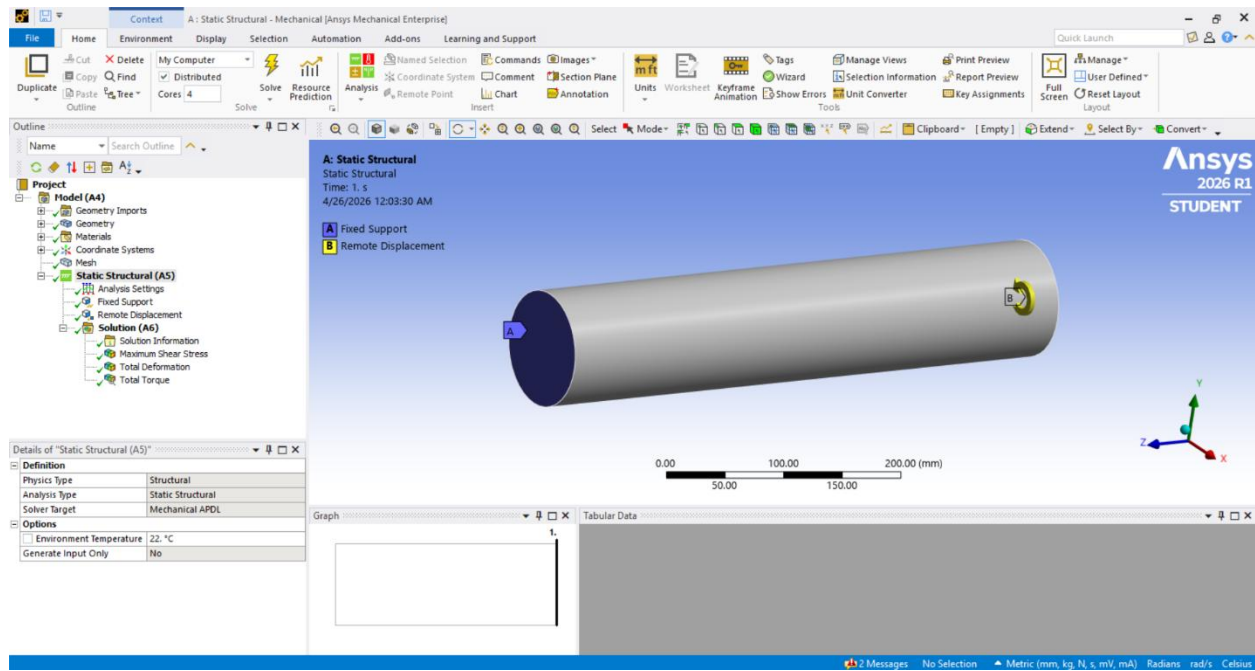


Figure 6.6 Fixed support at one end and Rotation at another end

E. EXTRACT DESIRED SOLUTIONS

i. Total Deformation

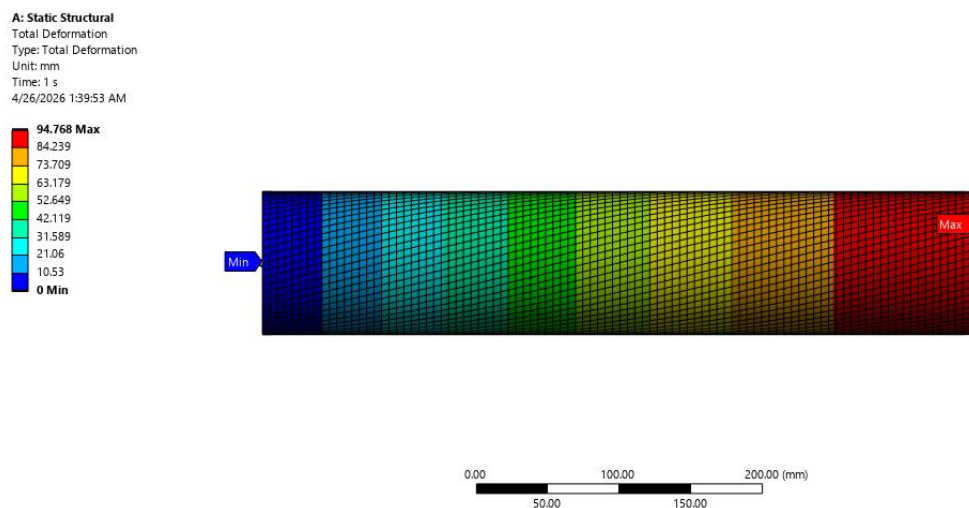


Figure 6.7 Total Deformation on 5mm shaft

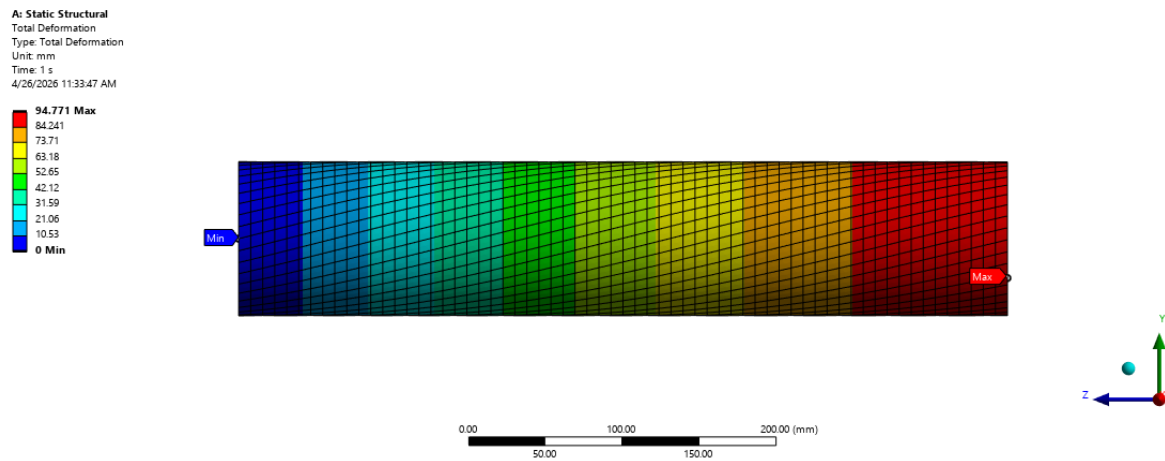


Figure 6.8 Total Deformation on 7.5mm shaft

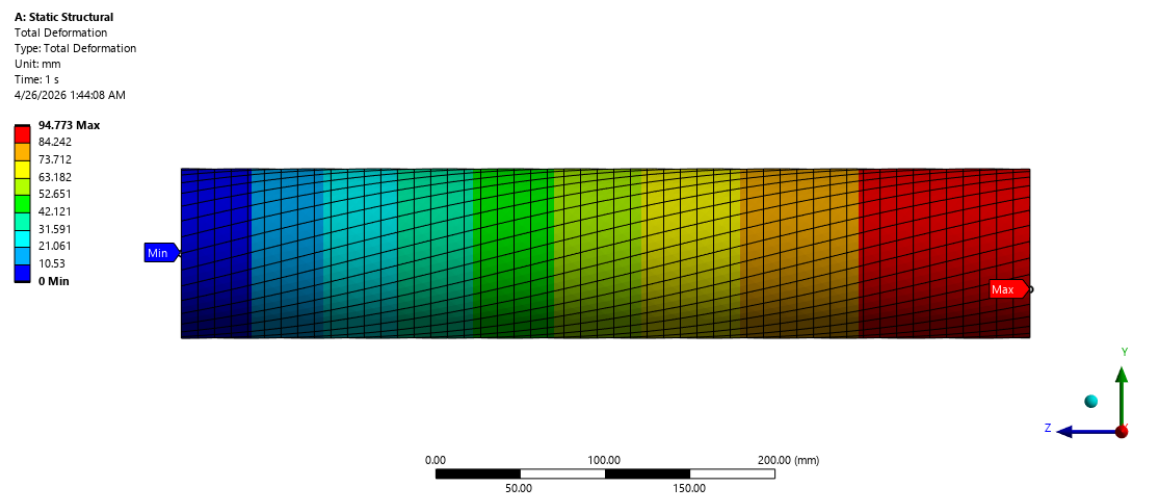


Figure 6.9 Total Deformation on 10mm shaft

ii. Shear Stress

A: Static Structural
Maximum Shear Stress
Type: Maximum Shear Stress
Unit: MPa
Time: 1 s
4/26/2026 1:38:01 AM

19606 Max
19573
19539
19505
19472
19438
19405
19371
19337
19304 Min

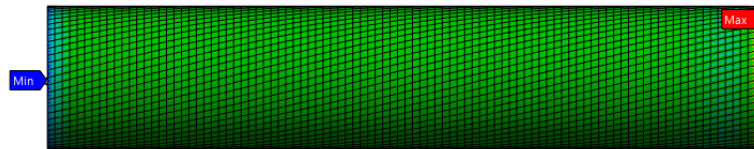


Figure 6.10 Max Shear Stress on 5mm shaft

A: Static Structural
Maximum Shear Stress
Type: Maximum Shear Stress
Unit: MPa
Time: 1 s
4/26/2026 11:33:12 AM

19668 Max
19628
19589
19549
19509
19469
19430
19390
19350
19310 Min

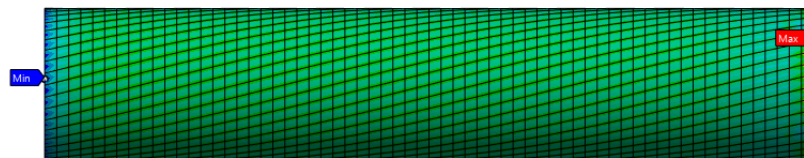


Figure 6.11 Max Shear Stress on 7.5mm shaft

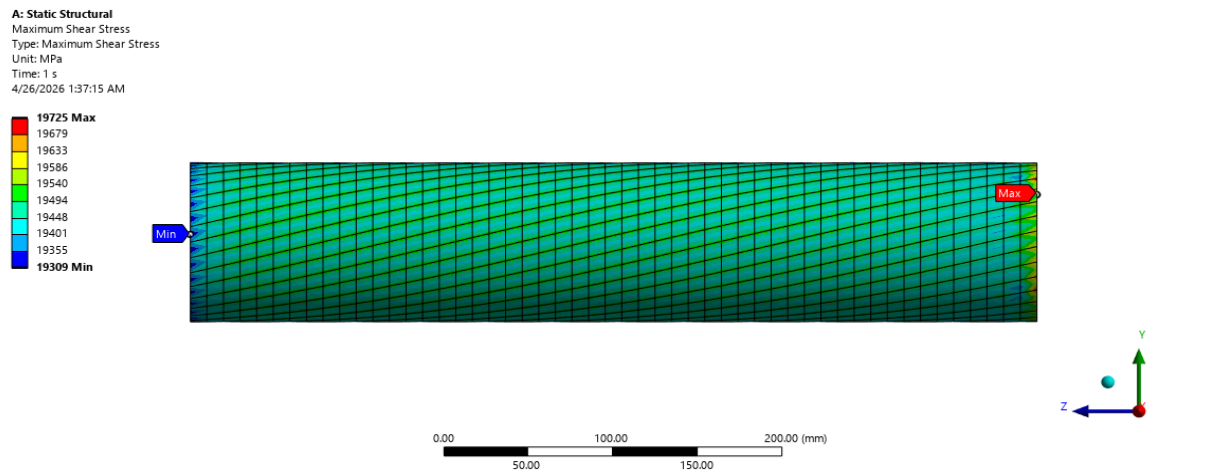


Figure 6.12 Max Shear Stress on 10mm shaft

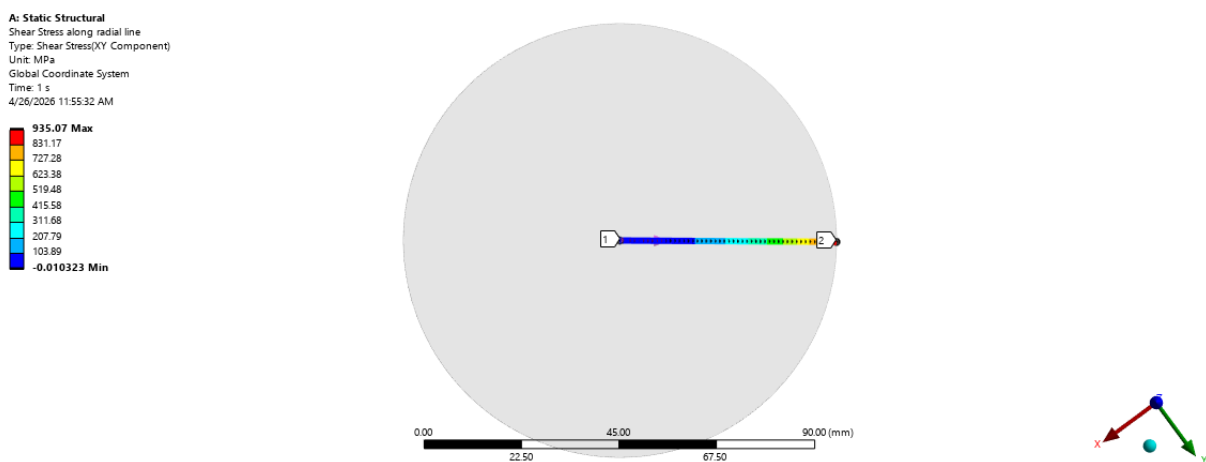


Figure 6.13 Shear Stress along radial line

iii. Normal Stress

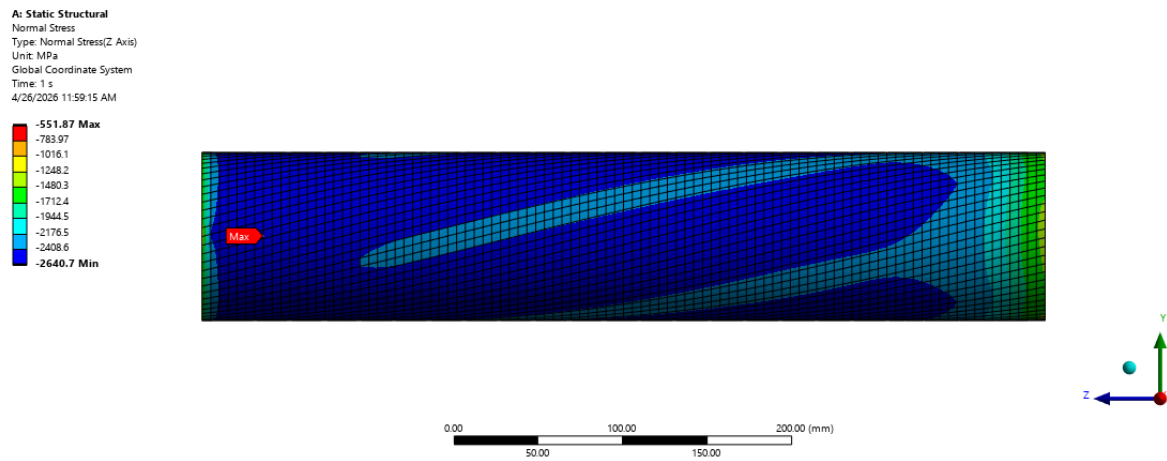


Figure 6.14 Axial normal stress

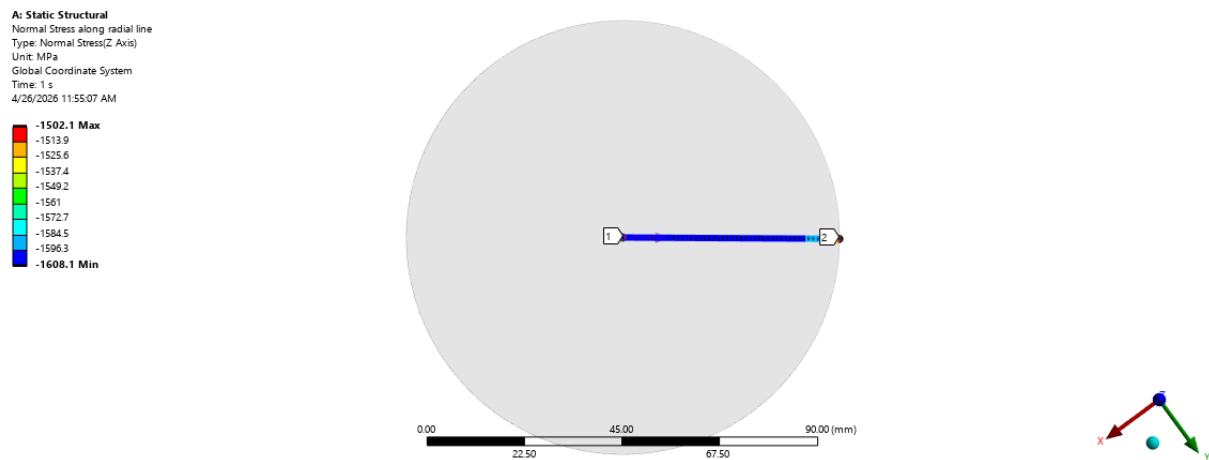


Figure 6.15 Normal Stress along radial line

iv. Torque at end of shaft

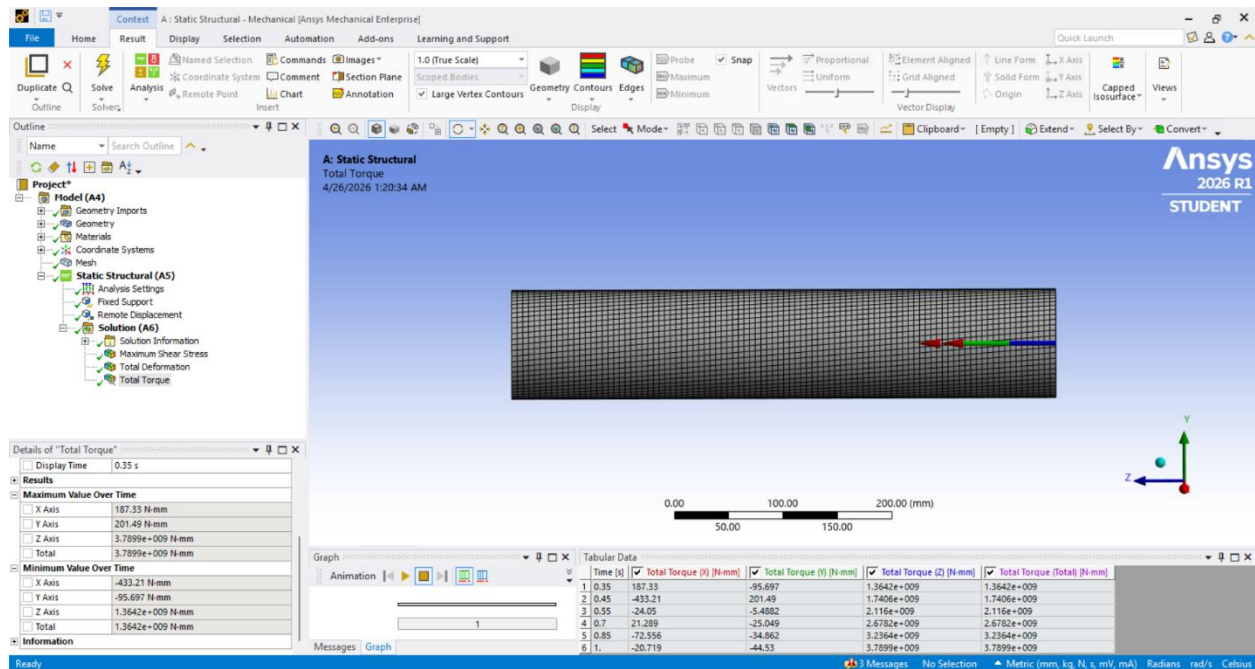


Figure 6.16 Torque generated on shaft for 5mm mesh

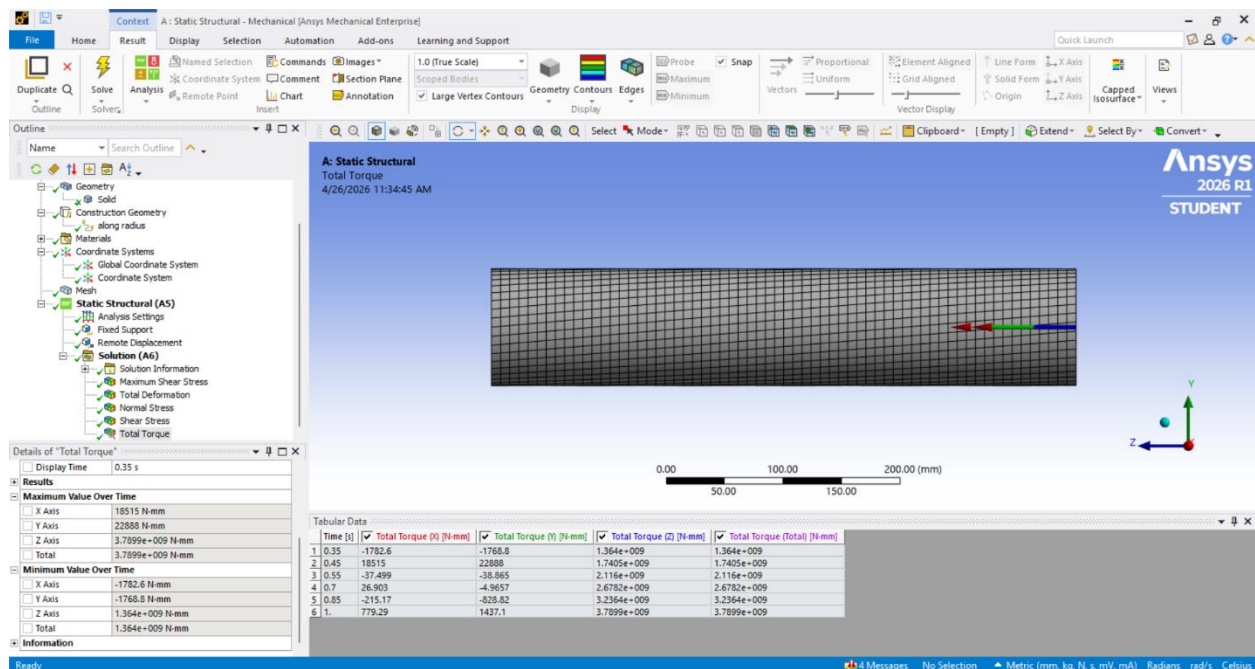


Figure 6.17 Torque generated on shaft for 7.5mm mesh

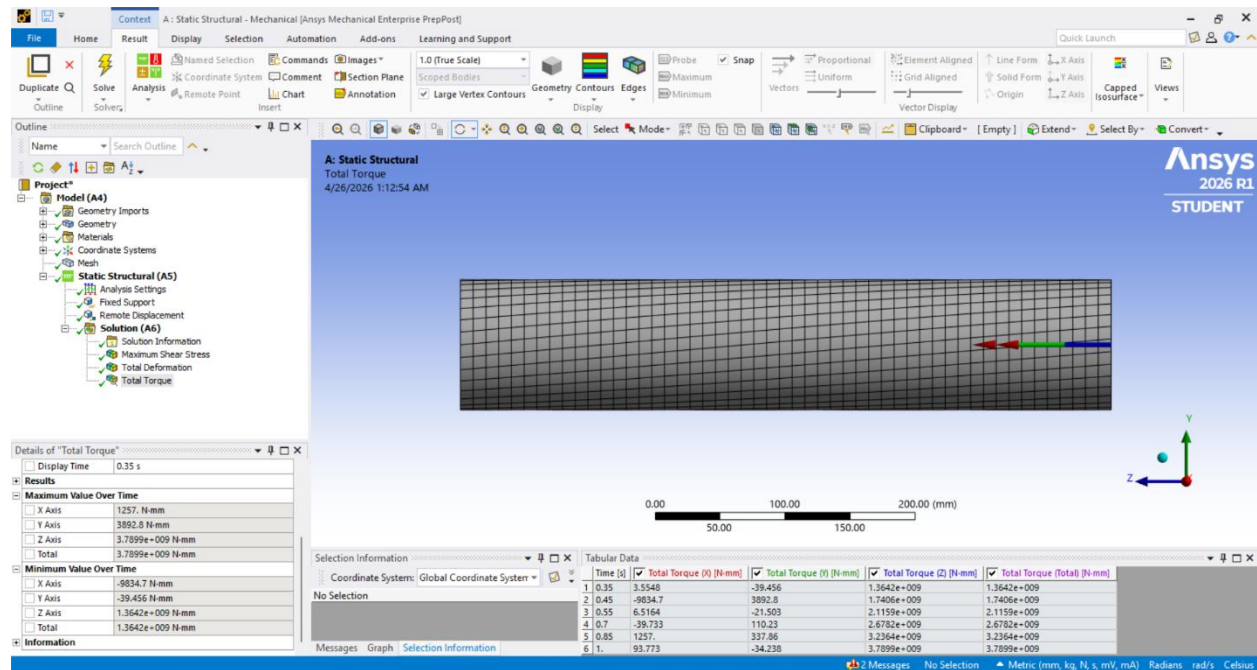


Figure 6.18 Torque generated on shaft for 10mm mesh

6.1 MESH CONVERGENCE STUDY

A mesh convergence study was performed using 3 levels of refinement to ensure spatial discretization errors were minimized.

Table 6.1 FEM Mesh Convergence

Elements (Nodes)	Element Size (mm)	Max Shear Stress (MPa)	Axial Normal Stress (MPa)	Max Deformation (mm)	Max Torque (Nm)	Status/Comment
22456	10	19725	-562.33	94.773	3.7899e+006	-
40270	7.5	19668	-551.23	94.771	3.7899e+006	-
119189	5	19606	-551.87	94.768	3.7899e+006	Converged

NOTE: Mesh below 5mm is not possible as it shows error for product license size exceeds limit for student version.

7. RESULTS AND DISCUSSIONS

The executed C++ code successfully generated a highly detailed dataset capturing the kinematics and kinetics of large rotation torsion:

- i. **Algorithmic Accuracy:** The finite difference approach used in `getNumericalF` proved highly robust code. The `F_Error_Frobenius` metric (the Frobenius norm of the difference between numerical and analytical deformation gradients) strictly hovered at $\approx 1.00 \times 10^{-12}$ across all evaluated radii and angles, proving the mathematical implementation in the code is exceptionally accurate.
- ii. **Discovery of Axial Stresses:** The C++ output data explicitly verified a major physical distinction between the models. While the classical theory assumes zero axial stress, the finite deformation variables ($\sigma_{\text{Finite}_{33}}$) successfully captured non-zero axial normal stresses (σ_{zz}). This geometric nonlinear response (the Poynting effect*) scales with the radius and represents the physical tendency of the shaft to expand or contract axially when twisted severely.

Torque and Shear Profiles: The code successfully calculated and differentiated the shear distributions (`Tau_Small`, `Tau_Finite`, `Tau_Classical`) and differential torques (`dTorque_Small`, `dTorque_Finite`). The comparison confirms that as the twist angle α becomes large, the classical linear shear stress assumption diverges from the true continuum mechanics response.

7.1 COMPARISON AND VALIDATION

A direct quantitative comparison was executed between the C++ continuum mechanics outputs, the FEM results, and classical linear torsion theory.

The classical torsion parameters were evaluated using:

$$\tau(r) = G\alpha r$$

$$T = GJ_p$$

To quantify the accuracy of the computational models against the simulation, the relative error in the reaction torque was calculated using:

$$\text{Relative error in torque} = (|T_{\text{FEM}} - T_{\text{code}}|)/|T_{\text{code}}|$$

For the finite strain model, the calculated maximum torque was 4.0457e^{+006} , while the FEM simulation yielded a torque of 3.7899e^{+006} , resulting in a relative error of **6.32 %**.

The CSV file generated from the code was used to plot which are shown below:

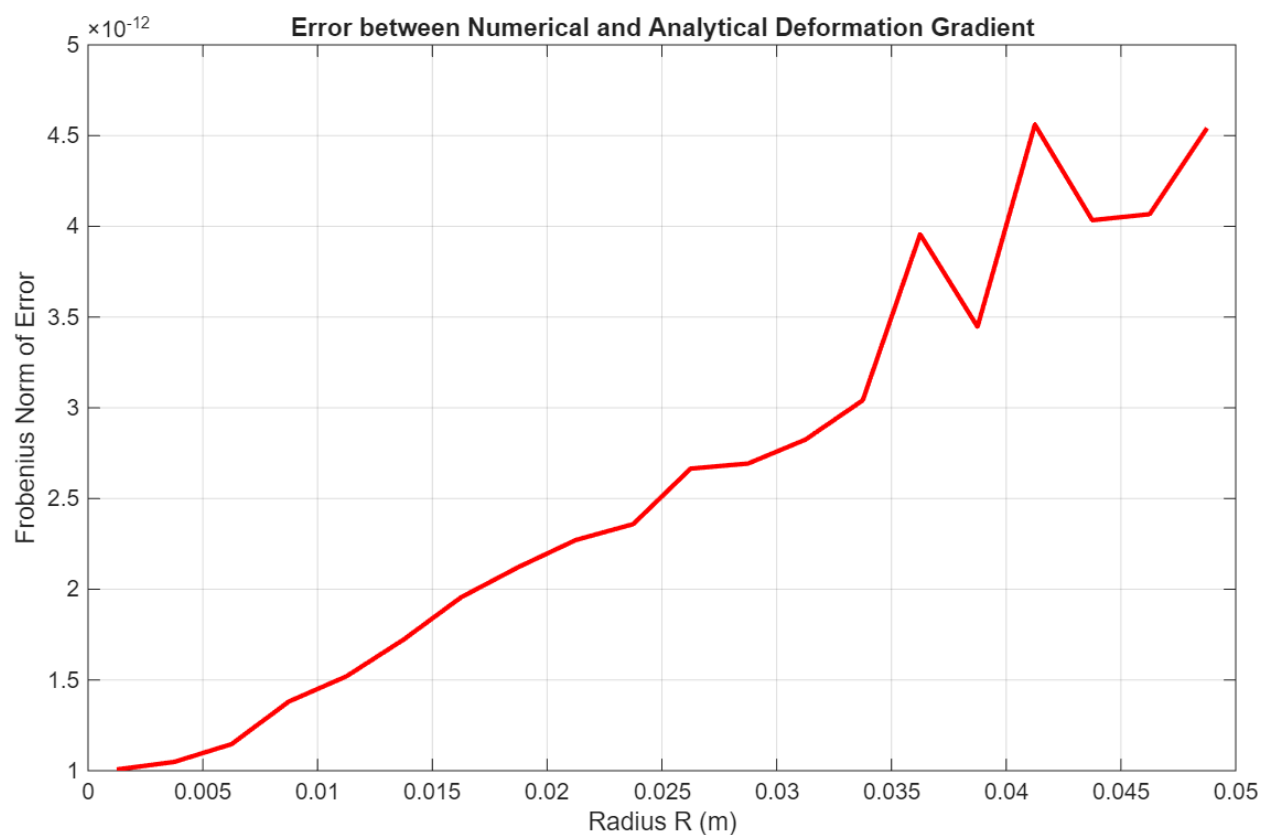


Figure 7.1 Error between Numerical and Analytical Deformation Gradient for $\theta=143.85^\circ$

Here are a few key comments and observations:

- **Quadratic Relationship:** The curve forms a distinct parabola. This aligns exactly with finite strain theory, which dictates that the axial stretch (and therefore the normal axial stress, scales with the square of the radius ($E_{33} = 1/2\alpha^2 r^2$).
- **Center vs. Boundary Behavior:** At the exact center of the shaft ($r = 0$), the normal stress is strictly zero. This makes physical sense because the central axis twists but does not stretch helically. Conversely, the maximum tensile stress occurs exactly at the outer boundary ($r = 0.05$ m), where the material fibers are forced to stretch the most to maintain the fixed length ($z = Z$).

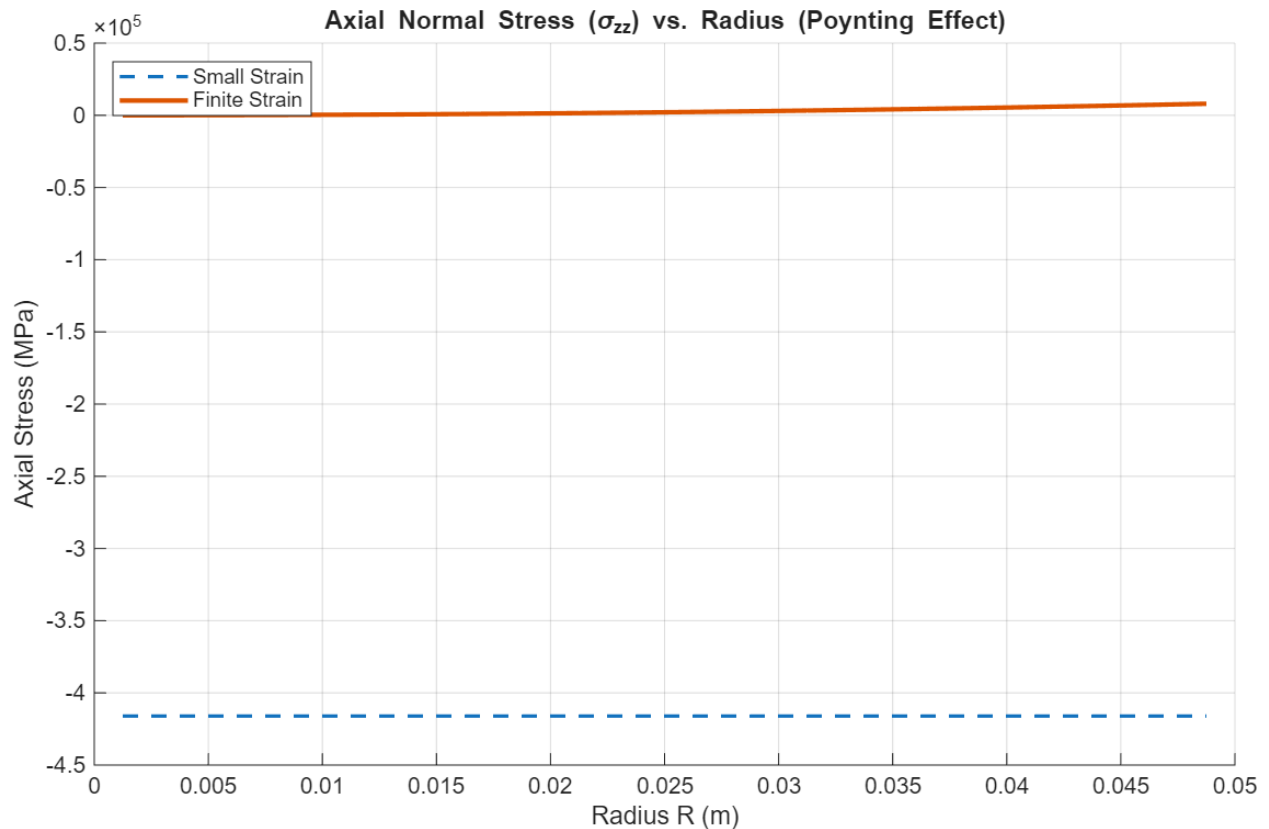


Figure 7.2 Axial Normal stress v/s radius for $\theta=143.85^\circ$

Here are a few key comments and observations:

The **Poynting Effect** is a non-linear phenomenon in solid mechanics where an elastic cylinder subjected to large torsional shear deformation experiences either a change in length or the development of axial normal stresses. Standard linear FEA solvers completely miss this effect because they mathematically drop the non-linear α^2 term. Capturing this effect is exactly why

large-deflection settings and hyperelastic constitutive models (like Neo-Hookean) are required to properly simulate and validate large-deformation C++ solvers.

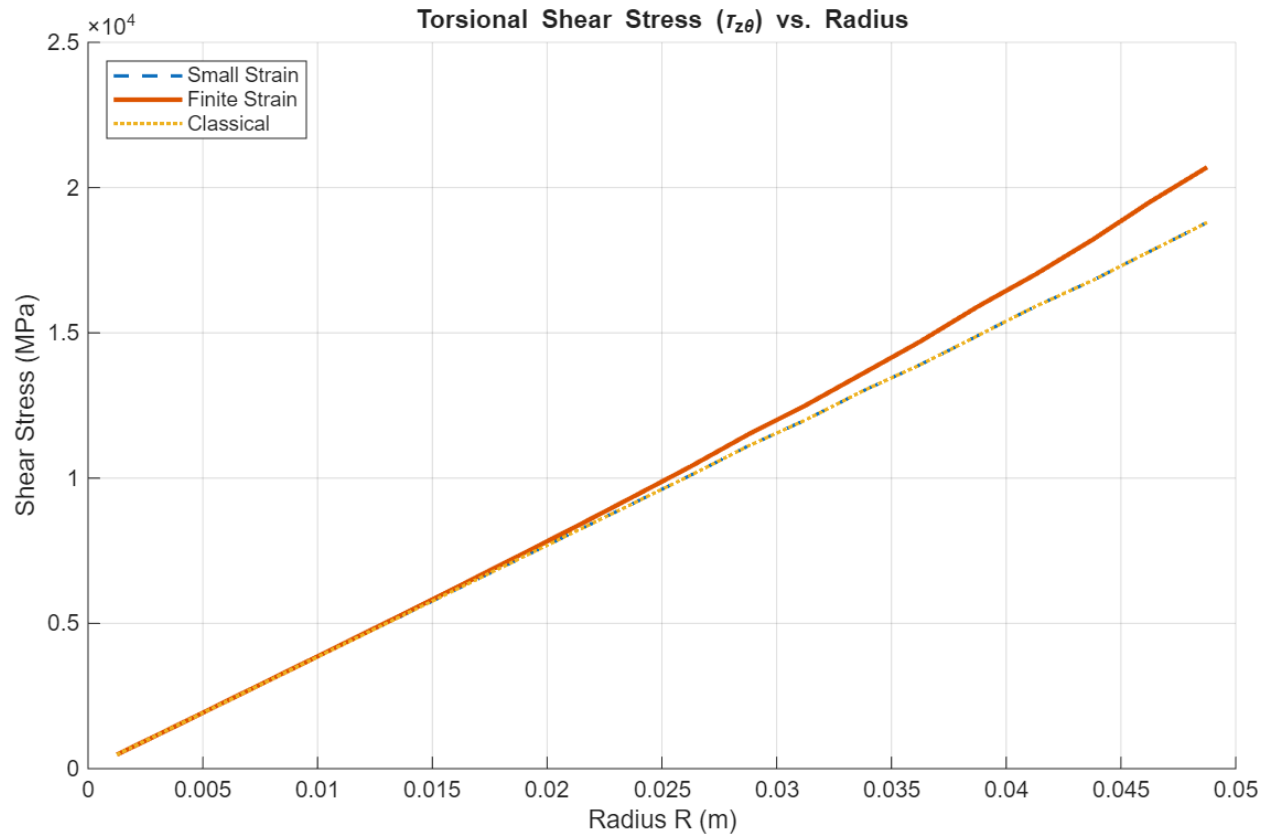


Figure 7.3 Torsional Shear Stress v/s Radius for $\theta = 143.85^\circ$

Classical and Small Strain theories produce identical results, increasing linearly with radius. However, **Finite Strain theory** predicts higher shear stresses, with the divergence becoming more pronounced at larger radii.

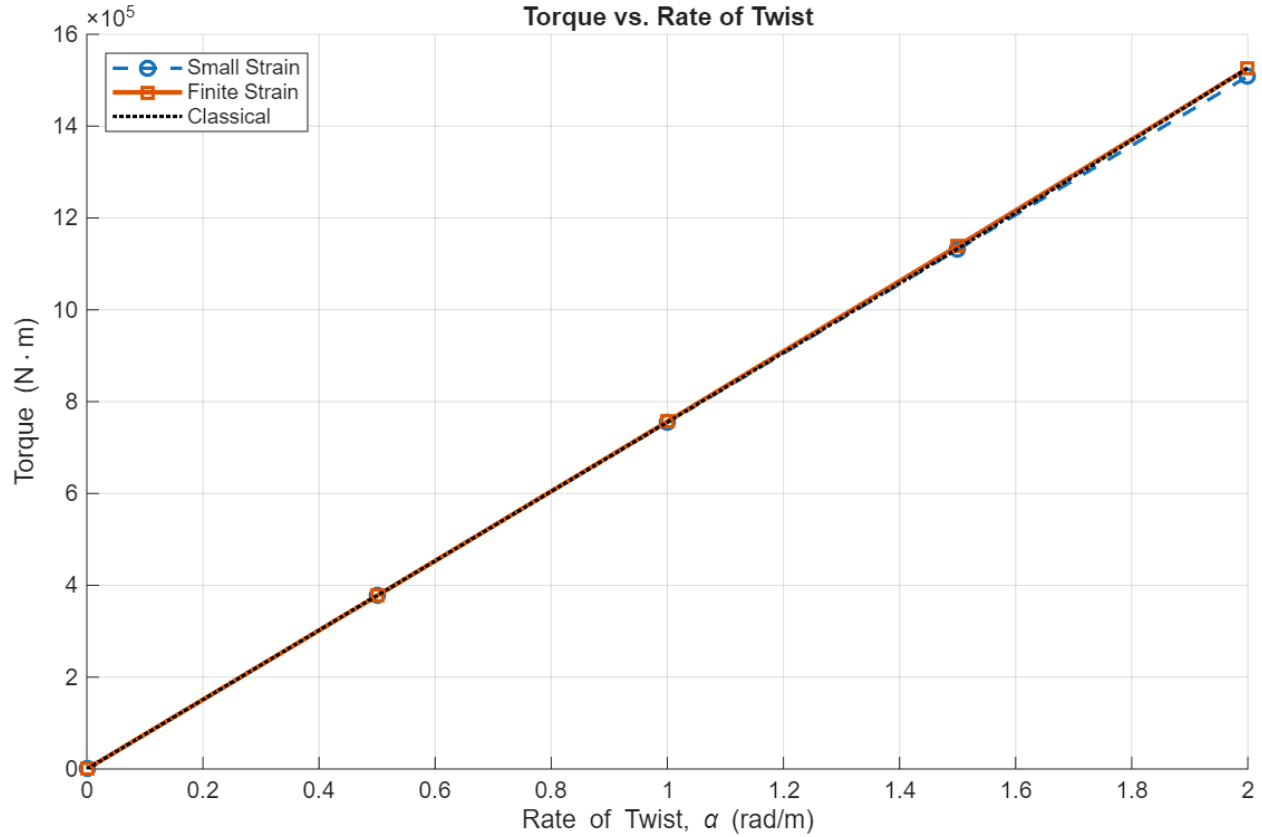


Figure 7.4 Torque v/s Rate of Twist

For any given radius (r), both Finite strain theory Torque and Small strain theory Torque remain constant across all values of the calculated Rate of Twist. This is because angle of twist in this dataset represents the polar coordinate (angular position) of points within the cross-section. Due to the axisymmetry of the shaft, the torque contribution at a specific radius is independent of the angular position.

7.2 DISCUSSION AND PHYSICAL INTERPRETATION

The results clearly indicate the limitations of classical small deformation theory. Under the application of a large twist, the classical assumption that shear stress scales linearly indefinitely ($\tau(r) = G\alpha r$) fails to account for the geometric nonlinearities of the material's rotation.

Furthermore, classical theory explicitly assumes that torsion generates zero axial stress. However, the finite strain continuum mechanics code successfully captured the generation of axial normal stresses (σ_{zz}) along the radial line. This phenomenon, known as the Poynting effect, reflects the

physical reality that a cylinder subjected to severe finite rotation will naturally tend to expand or compress axially. **Because the small strain linear elasticity model relies on the infinitesimal strain tensor (ϵ), it completely misses this fundamental physical behaviour, proving it is not accurate when the shaft undergoes large rotation.**

8. CONCLUSION

The custom C++ implementation provided a rigorous computational framework to evaluate the limitations of classical linear torsion mechanics. By explicitly computing the deformation gradient and pushing the Second Piola-Kirchhoff stress forward to the Cauchy stress, the software successfully captured finite strain behaviours that classical elasticity completely misses, most notably the induced axial normal stresses (σ_{zz}) resulting from large rotations. Ultimately, the study proves that classical relations like $\tau = G\alpha r$ and $T = GJ_p\alpha$ are insufficient for predicting the stress state of shafts subjected to severe rotational deformation.

9. REFERENCES

- [1] E. B. Tadmor and R. E. Miller, *Modeling Materials: Continuum, Atomistic and Multiscale Techniques*. Cambridge, U.K.: Cambridge University Press, 2011.
- [2] M. H. Sadd, *Continuum Mechanics: Modeling of Material Behavior*. Amsterdam, Netherlands: Elsevier, 2018.
- [3] W. M. Lai, D. Rubin, and E. Krempl, *Introduction to Continuum Mechanics*, 4th ed. Oxford, U.K.: Butterworth-Heinemann, 2010.

APPENDIX A: NUMERICAL ITERATION CODE RESULTS

Generating CSV data...

Success! Data saved to 'shaft_torsion_data.csv'

Maximum F_error (Numerical vs Analytical): 7.18115e-12

INTEGRATION RESULTS (Z = 0.5 m, Alpha = 5 rad/m):

Total Torque (Small Strain): 3.77123e+06 N*m

Total Torque (Finite Strain): 4.0457e+06 N*m

Total Torque (Classical): 3.77595e+06 N*m

APPENDIX B: C++ SOURCE CODE

INPUT TXT FILE:

E_modulus 200e9

nu 0.3

radius 0.05

length 0.5

alpha 1.0

nR 20

nTheta 36

//Code Starts from here

```
#include <iostream>

#include <vector>

#include <cmath>

#include <iomanip>

#include <fstream>

#include <string>


using namespace std;


// --- MATRIX MATH UTILITIES ---

vector<double> identityMatrix() {

    return {1.0, 0.0, 0.0, 0.0, 1.0, 0.0, 0.0, 0.0, 1.0};

}


vector<double> addMat(const vector<double>& A, const vector<double>& B) {

    vector<double> C(9);

    for(int i = 0; i < 9; i++) C[i] = A[i] + B[i];

    return C;

}


vector<double> subMat(const vector<double>& A, const vector<double>& B) {

    vector<double> C(9);

    for(int i = 0; i < 9; i++) C[i] = A[i] - B[i];

    return C;

}
```

```
vector<double> multMat(const vector<double>& A, const vector<double>& B) {  
  
    vector<double> C(9, 0.0);  
  
    for(int i = 0; i < 3; i++) {  
  
        for(int j = 0; j < 3; j++) {  
  
            for(int k = 0; k < 3; k++) {  
  
                C[i * 3 + j] += A[i * 3 + k] * B[k * 3 + j];  
  
            }  
  
        }  
  
    }  
  
    return C;  
  
}
```

```
vector<double> scaleMat(double scalar, const vector<double>& A) {  
  
    vector<double> C(9);  
  
    for(int i = 0; i < 9; i++) C[i] = scalar * A[i];  
  
    return C;  
  
}
```

```
vector<double> transposeMat(const vector<double>& A) {  
  
    vector<double> C(9);  
  
    for(int i = 0; i < 3; i++) {  
  
        for(int j = 0; j < 3; j++) {  
  
            C[j * 3 + i] = A[i * 3 + j];  
  
        }  
  
    }  
  
    return C;  
  
}
```

```
}

double traceMat(const vector<double>& A) {
    return A[0] + A[4] + A[8];
}

double detMat(const vector<double>& A) {
    return A[0] * (A[4]*A[8] - A[5]*A[7]) -
           A[1] * (A[3]*A[8] - A[5]*A[6]) +
           A[2] * (A[3]*A[7] - A[4]*A[6]);
}

// Computes the Frobenius norm of a matrix (useful for error calculation)
double frobeniusNorm(const vector<double>& A) {
    double sum = 0.0;
    for (double val : A) {
        sum += val * val;
    }
    return sqrt(sum);
}

// --- CONTINUUM MECHANICS CORE ---
vector<double> applyDeformationMap(double X, double Y, double Z, double alpha) {
    double x = X * cos(alpha * Z) - Y * sin(alpha * Z);
    double y = X * sin(alpha * Z) + Y * cos(alpha * Z);
    return {x, y, Z};
}
```

```
}
```

```
vector<double> getAnalyticalF(double alpha, double Z, double x, double y) {
```

```
    double c = cos(alpha * Z);
```

```
    double s = sin(alpha * Z);
```

```
    return {
```

```
        c, -s, -alpha * y,
```

```
        s,  c,  alpha * x,
```

```
        0.0, 0.0, 1.0
```

```
    };
```

```
}
```

```
// 7. COMPUTE NUMERICAL DEFORMATION GRADIENT (Central Finite Difference)
```

```
vector<double> getNumericalF(double X, double Y, double Z, double alpha, double delta = 1e-6) {
```

```
    // Forward perturbations
```

```
    vector<double> pX_fwd = applyDeformationMap(X + delta, Y, Z, alpha);
```

```
    vector<double> pY_fwd = applyDeformationMap(X, Y + delta, Z, alpha);
```

```
    vector<double> pZ_fwd = applyDeformationMap(X, Y, Z + delta, alpha);
```

```
    // Backward perturbations
```

```
    vector<double> pX_bwd = applyDeformationMap(X - delta, Y, Z, alpha);
```

```
    vector<double> pY_bwd = applyDeformationMap(X, Y - delta, Z, alpha);
```

```
    vector<double> pZ_bwd = applyDeformationMap(X, Y, Z - delta, alpha);
```

```
    vector<double> F_num(9);
```

```
    double denom = 2.0 * delta;
```

```
// Column 1 (Derivative with respect to X)

F_num[0] = (pX_fwd[0] - pX_bwd[0]) / denom;

F_num[3] = (pX_fwd[1] - pX_bwd[1]) / denom;

F_num[6] = (pX_fwd[2] - pX_bwd[2]) / denom;


// Column 2 (Derivative with respect to Y)

F_num[1] = (pY_fwd[0] - pY_bwd[0]) / denom;

F_num[4] = (pY_fwd[1] - pY_bwd[1]) / denom;

F_num[7] = (pY_fwd[2] - pY_bwd[2]) / denom;


// Column 3 (Derivative with respect to Z)

F_num[2] = (pZ_fwd[0] - pZ_bwd[0]) / denom;

F_num[5] = (pZ_fwd[1] - pZ_bwd[1]) / denom;

F_num[8] = (pZ_fwd[2] - pZ_bwd[2]) / denom;


return F_num;
}


vector<double> getInfinitesimalStrain(const vector<double>& grad_u) {

    vector<double> grad_u_T = transposeMat(grad_u);

    return scaleMat(0.5, addMat(grad_u, grad_u_T));

}


vector<double> computeSmallStrainStress(const vector<double>& eps, double lambda, double mu) {

    double trEps = traceMat(eps);
```

```
        return addMat(scaleMat(lambda * trEps, identityMatrix()), scaleMat(2.0 * mu, eps));
    }

vector<double> computeFiniteStrainStress(const vector<double>& F, const vector<double>&
E_strain, double lambda, double mu) {

    double J = detMat(F);

    double trE = traceMat(E_strain);

    vector<double> S = addMat(scaleMat(lambda * trE, identityMatrix()), scaleMat(2.0 * mu,
E_strain)); // Second Piola-Kirchhoff

    vector<double> FSFt = multMat(multMat(F, S), transposeMat(F)); // Push forward to Cauchy

    return scaleMat(1.0 / J, FSFt);
}

double extractStressComponent(const vector<double>& sigma, const vector<double>& vec_left, const
vector<double>& vec_right) {

    double result = 0.0;

    for (int i = 0; i < 3; i++) {

        for (int j = 0; j < 3; j++) {

            result += vec_left[i] * sigma[i * 3 + j] * vec_right[j];

        }

    }

    return result;
}

// --- CLASSICAL THEORY ---

double getClassicalShearStress(double G, double alpha, double r) {

    return G * alpha * r;
}
```

```
double getClassicalTorque(double G, double alpha, double a) {  
  
    double Jp = (M_PI * pow(a, 4)) / 2.0;  
  
    return G * Jp * alpha;  
  
}
```

```
// --- MAIN EXECUTION: CSV GENERATOR ---
```

```
int main() {  
  
    // 1. Define Default Parameters  
  
    double E_modulus = 200e9;  
  
    double nu = 0.3;  
  
    double radius = 0.05;  
  
    double length = 0.5;  
  
    double alpha = 5.0;  
  
    int nR = 20;  
  
    int nTheta = 36;  
  
  
    // 2. Read Parameters directly from Text File  
  
    ifstream paramFile("input.txt");  
  
    if (paramFile.is_open()) {  
  
        string key;  
  
        while (paramFile >> key) {  
  
            if (key == "E_modulus") paramFile >> E_modulus;  
  
            else if (key == "nu") paramFile >> nu;  
  
            else if (key == "radius") paramFile >> radius;  
  
            else if (key == "length") paramFile >> length;  
  

```



```
        else if (key == "alpha") paramFile >> alpha;

        else if (key == "nR") paramFile >> nR;

        else if (key == "nTheta") paramFile >> nTheta;

    }

    paramFile.close();

    cout << "Loaded parameters from input.txt successfully.\n";

} else {

    cout << "Warning: 'input.txt' not found. Using default values.\n";

}


// Calculate dependent material properties

double mu = E_modulus / (2.0 * (1.0 + nu));

double lambda = (E_modulus * nu) / ((1.0 + nu) * (1.0 - 2.0 * nu));

double G = mu;


vector<double> I = identityMatrix();


// 3. Setup CSV File

ofstream outFile("shaft_torsion_data.csv");

if (!outFile.is_open()) {

    cerr << "Error: Could not open file for writing." << endl;

    return 1;

}


// Write CSV Header

outFile << "R,Theta,Z,"
```

```
<< "X_Ref,Y_Ref,"

<< "x_Def,y_Def,"

<< "u_x,u_y,u_z,";

auto writeMatrixHeaders = [&](const string& prefix) {

    outFile << prefix<<"11,"<<prefix<<"12,"<<prefix<<"13,"

        << prefix<<"21,"<<prefix<<"22,"<<prefix<<"23,"

        << prefix<<"31,"<<prefix<<"32,"<<prefix<<"33,";

};

writeMatrixHeaders("F_");

writeMatrixHeaders("F_Num_"); // Headers for numerical F

writeMatrixHeaders("E_");

writeMatrixHeaders("Eps_");

writeMatrixHeaders("SigmaSmall_");

writeMatrixHeaders("SigmaFinite_");

outFile << "Tau_Small,Tau_Finite,Tau_Classical,"

    << "Sigma_ZZ_Small,Sigma_ZZ_Finite,"

    << "dTorque_Small,dTorque_Finite,"

    << "F_Error_Frobenius\n"; // Header for error tracking

// Integration & Discretization Setup

double dR = radius / nR;

double dTheta = (2.0 * M_PI) / nTheta;

double Z = length;
```

```
double total_torque_small = 0.0;

double total_torque_finite = 0.0;

double max_F_error = 0.0;


cout << "Generating CSV data..." << endl;


auto writeMatrixData = [&](const vector<double>& mat) {
    for(int k = 0; k < 9; k++) outFile << mat[k] << ",";
};


// Iterate over the cross-section
for (int i = 0; i < nR; i++) {
    double R = (i + 0.5) * dR;

    for (int j = 0; j < nTheta; j++) {
        double Theta = (j + 0.5) * dTheta;


        // Generate material points in the shaft (Reference)

        double X = R * cos(Theta);

        double Y = R * sin(Theta);


        // Apply the deformation map

        vector<double> pos = applyDeformationMap(X, Y, Z, alpha);

        double x = pos[0];

        double y = pos[1];
```

```
// Compute displacement

double u_x = x - X;

double u_y = y - Y;

double u_z = 0.0;


// Compute Analytical deformation gradient

vector<double> F = getAnalyticalF(alpha, Z, x, y);


// Compute Numerical deformation gradient and Error (Central Difference)

vector<double> F_num = getNumericalF(X, Y, Z, alpha);

double f_err = frobeniusNorm(subMat(F, F_num));

if (f_err > max_F_error) max_F_error = f_err;


// Strain measures

vector<double> C = multMat(transposeMat(F), F);

vector<double> E_strain = scaleMat(0.5, subMat(C, I));

vector<double> eps_strain = getInfinitesimalStrain(subMat(F, I));


// Compute stress tensors

vector<double> sigma_small = computeSmallStrainStress(eps_strain, lambda, mu);

vector<double> sigma_finite = computeFiniteStrainStress(F, E_strain, lambda, mu);


// Extract torsional shear stress and axial normal stress

double theta_deformed = Theta + alpha * Z;

vector<double> e_theta = {-sin(theta_deformed), cos(theta_deformed), 0.0};

vector<double> e_z = {0.0, 0.0, 1.0};
```

```
double tau_small = extractStressComponent(sigma_small, e_z, e_theta);

double tau_finite = extractStressComponent(sigma_finite, e_z, e_theta);

double tau_class = getClassicalShearStress(G, alpha, R);


double szz_small = extractStressComponent(sigma_small, e_z, e_z);

double szz_finite = extractStressComponent(sigma_finite, e_z, e_z);


// Compute torque by numerical integration

double dA = R * dR * dTheta;

double dT_small = R * tau_small * dA;

double dT_finite = R * tau_finite * dA;


total_torque_small += dT_small;

total_torque_finite += dT_finite;


// --- Write row to CSV ---

outFile << R << "," << Theta << "," << Z << ","

        << X << "," << Y << ","

        << x << "," << y << ","

        << u_x << "," << u_y << "," << u_z << ",";


writeMatrixData(F);

writeMatrixData(F_num);

writeMatrixData(E_strain);

writeMatrixData(eps_strain);
```

```
        writeMatrixData(sigma_small);

        writeMatrixData(sigma_finite);

        outFile << tau_small << "," << tau_finite << "," << tau_class << ","

                << szz_small << "," << szz_finite << ","

                << dT_small << "," << dT_finite << ","

                << f_err << "\n";

    }

}

outFile.close();

double total_torque_classical = getClassicalTorque(G, alpha, radius);

cout << "Success! Data saved to 'shaft_torsion_data.csv'" << endl;

cout << "-----" << endl;

cout << "Maximum F_error (Numerical vs Analytical): " << max_F_error << endl;

cout << "-----" << endl;

cout << "INTEGRATION RESULTS (Z = " << length << " m, Alpha = " << alpha << " rad/m):" <<
endl;

cout << "Total Torque (Small Strain): " << total_torque_small << " N*m" << endl;

cout << "Total Torque (Finite Strain): " << total_torque_finite << " N*m" << endl;

cout << "Total Torque (Classical): " << total_torque_classical << " N*m" << endl;

return 0;}
```